

This member-only story is on us. [Upgrade](#) to access all of Medium.

★ Member-only story

# Running ONNX-Based LLM Router in Deno: A Practical Guide



Nima Mahmoudi · [Follow](#)

Published in ITNEXT

9 min read · Jan 13, 2025

Listen

Share

More

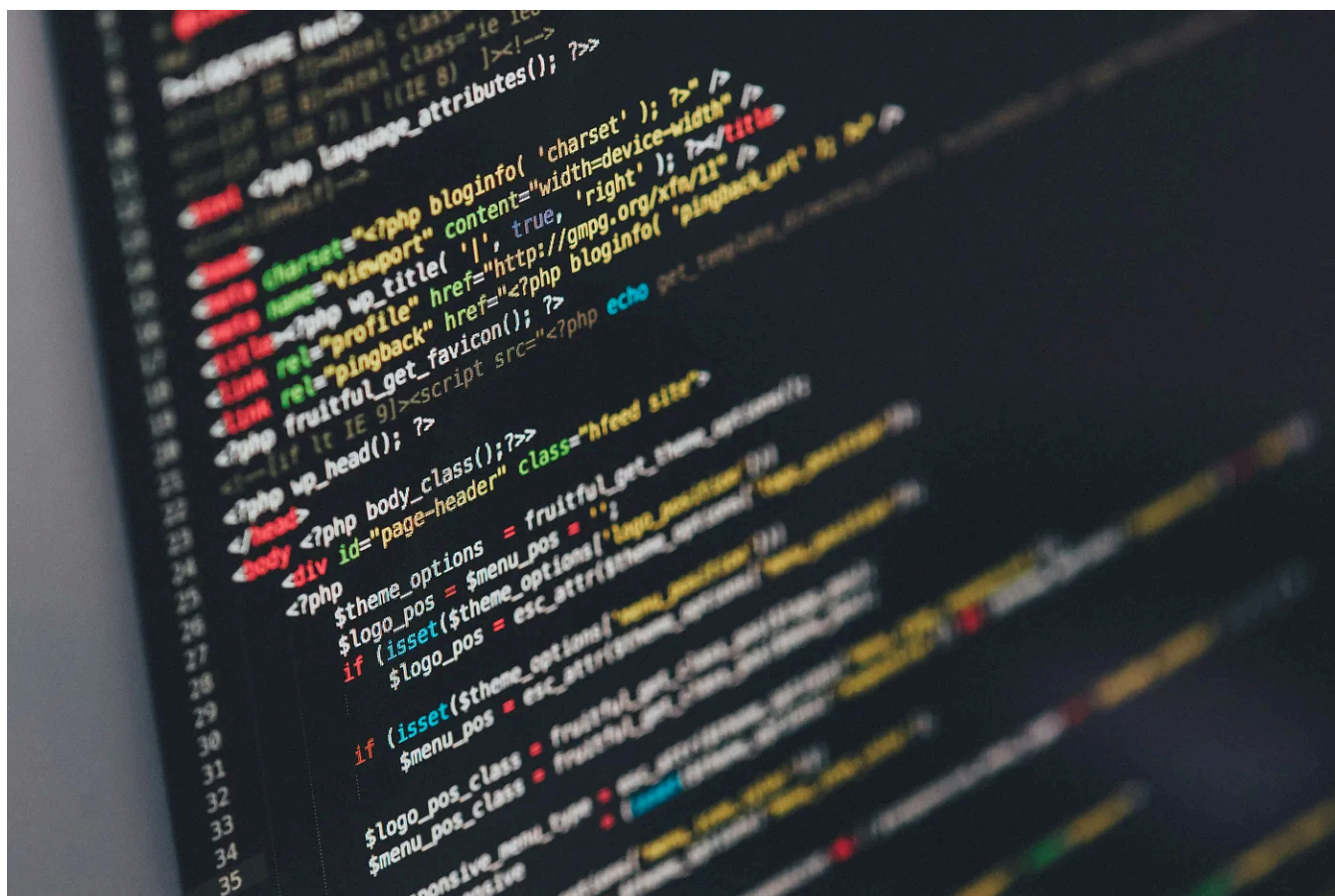


Photo by [Ilya Pavlov](#) on [Unsplash](#)

In a [previous article](#), we explored building an LLM router using OpenAI embeddings as a cost-effective and scalable solution. Now, we'll dive into how to deploy and serve this router using Deno, a modern JavaScript runtime. This implementation transforms our trained ONNX model into a production-ready service that can efficiently route queries between different LLMs or be used to find the most efficient prompt for the given query.

## What is Deno?

[Deno](#) is a modern runtime for JavaScript and TypeScript, created by Ryan Dahl, the original creator of Node.js. It was built to address some of the architectural limitations and security concerns present in Node.js. Unlike Node.js, Deno:

- Has built-in TypeScript support
- Uses ES Modules by default instead of CommonJS
- Implements secure defaults (explicit permissions required for file/network access)
- Includes a comprehensive standard library
- Has built-in developer tools like a formatter and test runner

## ONNX Runtime

[ONNX](#) (Open Neural Network Exchange) is an open standard for representing machine learning models. It acts as a bridge between different ML frameworks, allowing models trained in one framework to be deployed in another. Key benefits include:

- Framework interoperability (PyTorch, TensorFlow, and SciKitLearn are supported)
- Runtime optimization for different hardware
- Standardized model format
- Wide ecosystem support

## Getting Started with ONNX on Deno

Let's look at a real-world example of running an ONNX model in Deno. In this case, we'll use a CLIP (Contrastive Language-Image Pre-Training) model that's been converted to ONNX format ([reference](#)).

```
import * as ort from "npm:onnxruntime-node";

// URL of the remote ONNX model
const MODEL_URL = "https://huggingface.co/rocca/openai-clip-js/resolve/main/clip.onnx";

console.log("Downloading model...");
const response = await fetch(MODEL_URL);
const modelArrayBuffer = await response.arrayBuffer();

// Create session from in-memory buffer
let onnxImageSession = await ort.InferenceSession.create(new Uint8Array(modelArrayBuffer));

// Now run inference as before
for (let i = 0; i < 5; i++) {
  let data = new Float32Array(3 * 224 * 224).map(() => Math.random() - 0.5);
  const feeds = { 'input': new ort.Tensor('float32', data, [1,3,224,224]) };
  let t = performance.now();
  console.log("Starting inference...");
  const results = await onnxImageSession.run(feeds);
  console.log(`Finished inference in ${performance.now() - t}ms`);
}
```

Let's break down what's happening in this code:

1. We import ONNX Runtime from npm using Deno's npm compatibility layer.
2. Download a pre-trained CLIP model from HuggingFace in ONNX format.
3. Create an inference session from the downloaded model.
4. Run multiple inferences with random input data to test performance.
5. Measure and log the inference time for each run.

To run this code, you can either install Deno or use Docker. Since I like running everything in Docker, here is an example Docker compose file to run this script:

```
services:
  deno:
    container_name: deno_container
    image: denoland/deno
    volumes:
```

```
- ./main.ts:/app/main.ts
command: run --allow-ffi --allow-net /app/main.ts
```

Unlike traditional Node.js applications, Deno eliminates the need for local dependency storage via handling package resolution and caching.

You can run the script by running `docker compose up` and you should get a response like this:

```
Attaching to deno_public
deno_public | Download https://registry.npmjs.org/onnxruntime-node
...
deno_public | Download https://registry.npmjs.org/color-name/-/color-name-1.1.
deno_public | Downloading model...
deno_public | Starting inference...
deno_public | Finished inference in 28.2695239999999365ms
deno_public | Starting inference...
deno_public | Finished inference in 21.274706000000015ms
deno_public | Starting inference...
deno_public | Finished inference in 20.9310930000001056ms
deno_public | Starting inference...
deno_public | Finished inference in 21.301003999999901ms
deno_public | Starting inference...
deno_public | Finished inference in 21.1111110000000164ms
deno_public exited with code 0
```

As you can see, we are now able to load the ONNX model in a session and make inferences on it within 20–30ms.

## Serving An LLM Router API using Deno

Now that we have a working environment with ONNX and deno, let's expand on it to serve our LLM router built on top of OpenAI embeddings (for more information about this model, refer to [this article](#)).

Here is the code that loads and serves both EmbeddingNet models we developed (similar to our `express.js` code [here](#)):

```
// main.ts
import { Application, Router, Context } from "https://deno.land/x/oak/mod.ts";
import * as ort from "npm:onnxruntime-node";
```

```
import { OpenAI } from "npm:openai";

// -----
// Configuration object for models
// -----
const modelConfig: Record<string, {
  classMap: string[],
  modelPath: string,
  modelName: string,
  embeddingModel: string
}> = {
  banking77: {
    classMap: [
      "activate_my_card",
      "age_limit",
      "apple_pay_or_google_pay",
      "atm_support",
      "automatic_top_up",
      "balance_not_updated_after_bank_transfer",
      "balance_not_updated_after_cheque_or_cash_deposit",
      "beneficiary_not_allowed",
      "cancel_transfer",
      "card_about_to_expire",
      "card_acceptance",
      "card_arrival",
      "card_delivery_estimate",
      "card_linking",
      "card_not_working",
      "card_payment_fee_charged",
      "card_payment_not_recognised",
      "card_payment_wrong_exchange_rate",
      "card_swallowed",
      "cash_withdrawal_charge",
      "cash_withdrawal_not_recognised",
      "change_pin",
      "compromised_card",
      "contactless_not_working",
      "country_support",
      "declined_card_payment",
      "declined_cash_withdrawal",
      "declined_transfer",
      "direct_debit_payment_not_recognised",
      "disposable_card_limits",
      "edit_personal_details",
      "exchange_charge",
      "exchange_rate",
      "exchange_via_app",
      "extra_charge_on_statement",
      "failed_transfer",
      "fiat_currency_support",
      "get_disposable_virtual_card",
      "get_physical_card",
      "getting_spare_card",
```

```

        "getting_virtual_card",
        "lost_or_stolen_card",
        "lost_or_stolen_phone",
        "order_physical_card",
        "passcode_forgotten",
        "pending_card_payment",
        "pending_cash_withdrawal",
        "pending_top_up",
        "pending_transfer",
        "pin_blocked",
        "receiving_money",
        "Refund_not_showing_up",
        "request_refund",
        "reverted_card_payment?",
        "supported_cards_and_currencies",
        "terminate_account",
        "top_up_by_bank_transfer_charge",
        "top_up_by_card_charge",
        "top_up_by_cash_or_cheque",
        "top_up_failed",
        "top_up_limits",
        "top_up_reverted",
        "topping_up_by_card",
        "transaction_charged_twice",
        "transfer_fee_charged",
        "transfer_into_account",
        "transfer_not_received_by_recipient",
        "transfer_timing",
        "unable_to_verify_identity",
        "verify_my_identity",
        "verify_source_of_funds",
        "verify_top_up",
        "virtual_card_not_working",
        "visa_or_mastercard",
        "why_verify_identity",
        "wrong_amount_of_cash_received",
        "wrong_exchange_rate_for_cash_withdrawal",
    ],
    modelPath: "https://thunderous-taiyaki-7420d0.netlify.app/banking77__text-e",
    modelName: "banking77",
    embeddingModel: "text-embedding-3-large",
},
"llm-router": {
    classMap: [
        "small_llm",
        "large_llm",
    ],
    modelPath: "https://thunderous-taiyaki-7420d0.netlify.app/llmrouter__text-e",
    modelName: "llm-router",
    embeddingModel: "text-embedding-3-large",
},
};

```

```

// -----
// Store loaded model sessions
// -----
const sessions: Record<string, ort.InferenceSession> = {};

// -----
// load remote model
// -----
async function loadONNXModel(url: string) {
    console.log("Downloading model from remote location...");
    const response = await fetch(url);
    if (!response.ok) {
        throw new Error(`Error downloading model from signed URL: ${response.status}`);
    }

    // Convert response to ArrayBuffer
    const modelArrayBuffer = await response.arrayBuffer();

    // Create ONNX session from in-memory buffer
    const session = await ort.InferenceSession.create(new Uint8Array(modelArrayBuffer));
    console.log(`Loaded model from ${url} successfully`);

    return session;
}

// -----
// Load all ONNX models asynchronously
// -----
(async () => {
    for (const [modelId, config] of Object.entries(modelConfig)) {
        try {
            sessions[modelId] = await loadONNXModel(config.modelPath);
            console.log(`${config.modelName} ONNX model loaded successfully`);
        } catch (error) {
            console.error(`Failed to load ${config.modelName} ONNX model:`, error);
        }
    }
})();

// -----
// Helper function to calculate softmax
// -----
function softmax(logits: number[]): number[] {
    const maxLogit = Math.max(...logits); // For numerical stability
    const expLogits = logits.map((logit) => Math.exp(logit - maxLogit));
    const sumExpLogits = expLogits.reduce((sum, value) => sum + value, 0);
    return expLogits.map((value) => value / sumExpLogits);
}

// -----
// Create the router
// -----

```

```

const router = new Router();

// Health check endpoint
router.get("/healthz", (context: Context) => {
  context.response.status = 200;
  context.response.body = { status: "healthy" };
});

// Endpoint to get predictions
router.post("/api/embeddingnet/:model/predict", async (context: Context) => {
  try {
    const model = context.params.model;
    if (!model || !modelConfig[model]) {
      context.response.status = 400;
      context.response.body = { error: "Invalid model name provided" };
      return;
    }
    if (!sessions[model]) {
      context.response.status = 503;
      context.response.body = { error: `Model ${model} is not loaded or unavailable` };
      return;
    }

    // Extract API key from X-OpenAI-API-Key header
    const apiKey = context.request.headers.get("x-openai-api-key");
    if (!apiKey) {
      context.response.status = 401;
      context.response.body = { error: "API key is required in the x-openai-api-key" };
      return;
    }

    // Parse JSON body
    const body = context.request.body({ type: "json" });
    const { text } = await body.value;
    if (!text) {
      context.response.status = 400;
      context.response.body = { error: "Text input is required" };
      return;
    }

    // Initialize OpenAI client
    const openai = new OpenAI({ apiKey });

    // -----
    // 1. Get embeddings from OpenAI
    // -----
    const openAITimeStart = Date.now();
    const embeddingResponse = await openai.embeddings.create({
      model: modelConfig[model].embeddingModel,
      input: text,
    });
    const openAITimeEnd = Date.now();
    console.log(`Time taken for OpenAI embeddings: ${openAITimeEnd - openAITimeStart}ms`);
  } catch (error) {
    context.response.status = 500;
    context.response.body = { error: "Internal server error" };
  }
});

```

```

const embedding = embeddingResponse.data[0].embedding;

// -----
// 2. Prepare ONNX input
// -----
const inputTensor = new ort.Tensor("float32", embedding, [1, embedding.length]);
// Use the first input name of the session
const inputKey = sessions[model].inputNames[0];
const inputObject: Record<string, ort.Tensor> = {};
inputObject[inputKey] = inputTensor;

// -----
// 3. Run ONNX model inference
// -----
const onnxTimeStart = Date.now();
const results = await sessions[model].run(inputObject);
const onnxTimeEnd = Date.now();
console.log(`Time taken for ONNX model inference: ${onnxTimeEnd - onnxTimeStart}ms`);

// Assuming single output tensor
const outputKey = Object.keys(results)[0];
const tensor = results[outputKey];
const prob_logits = Array.from(tensor.data as Float32Array);

// Calculate probabilities
const probabilities = softmax(prob_logits);

// Get the predicted class and probability score
const predictedClassIndex = prob_logits.indexOf(Math.max(...prob_logits));
const probabilityScore = probabilities[predictedClassIndex];
const predictedClass = modelConfig[model].classMap[predictedClassIndex];

context.response.body = {
  predictedClassIndex,
  predictedClass,
  probabilityScore,
  probabilities,
};
} catch (error) {
  console.error("Error in /api/embeddingnet/:model/predict:", error);
  context.response.status = 500;
  context.response.body = { error: "Internal server error" };
}
});

// -----
// Create and start the Oak server
// -----
const app = new Application();
app.use(router.routes());
app.use(router.allowedMethods());

```

```
// const PORT = Deno.env.get("PORT") ?? "3000";
const PORT = "3000";
console.log(`Server running on http://localhost:${PORT}`);
await app.listen({ port: parseInt(PORT) });
```

Let's break down the code piece by piece. Here we are using Oak, which is a middleware framework for Deno that simplifies API development. We are also using ONNX runtime which enables fast and efficient inference of our exported router model. The server supports multiple routing models through a flexible configuration system:

```
const modelConfig: Record<string, {
  classMap: string[],
  modelPath: string,
  modelName: string,
  embeddingModel: string
}>js
```

To load each model, we go through these steps: 1) Download the ONNX router model from the specified location, passing the buffer to the ONNX runtime library, alternatively, you could pass a local path to the model file, but I wanted to keep the code standalone without needing additional files; 2) Initialize an ONNX Runtime session; and 3) Keep the model in memory for fast inference.

This happens automatically when the server starts:

```
(async () => {
  for (const [modelId, config] of Object.entries(modelConfig)) {
    try {
      sessions[modelId] = await loadONNXModel(config.modelPath);
      console.log(`${config.modelName} ONNX model loaded successfully`);
    } catch (error) {
      console.error(`Failed to load ${config.modelName} ONNX model:`, error);
    }
  }
})();
```

When a new request comes in for inference, we go through these steps:

1. Get the OpenAI API key and input text from the request.
2. Fetch the OpenAI embeddings for the text using the API key provided.
3. Prepare the embedding tensor for inference.
4. Run the embedding through the ONNX model to get predicted logits.
5. Apply softmax to get routing probabilities.
6. Return the recommended path with confidence scores per path.

You can use a Python script like the following to send a request to the server:

```
import requests
import json

url = "http://localhost:3000/api/embeddingnet/banking77/predict"

payload = json.dumps({
    "text": "when does my credit card arrive?"
})
headers = {
    'x-openai-api-key': YOUR_OPENAI_API_KEY,
    'Content-Type': 'application/json'
}

response = requests.request("POST", url, headers=headers, data=payload)

print(response.json())
```

And here's what a typical response looks like:

```
{
  "predictedClassIndex": 12,
  "predictedClass": "card_delivery_estimate",
  "probabilityScore": 0.9839192171160827,
  "probabilities": [
    0.0001875726020599338,
    ...
  ]
}
```

Notice that the text in the “predictedClass” comes from the model config’s classMap (mapping between the prediction index and the text for that class).

Now that our API server is ready to be deployed, let’s update our Docker compose file to work with the new script:

```
services:
  api-server:
    container_name: deno_container
    image: denoland/deno
    ports:
      - "3000:3000"
    volumes:
      - ./main.ts:/app/main.ts
    command: run --allow-ffi --allow-net /app/main.ts
```

Notice how this time we are exposing port 3000 of the container. We are now ready to deploy this script on a server. For personal deployments or side projects, I suggest simply using Docker Compose with this container alongside Caddy (for automatic SSL certificates), or alternatively, using GCP CloudRun.

## What about Deno Deploy?

Deno Deploy promises to be the “Hassle-free platform for serverless JavaScript applications”. It is trying to be a single command deployment solution and would have been the perfect platform to deploy our code to, but at the time of writing, there seems to be some incompatibility with the ONNX node runtime library which leads to this error:

```
Error: This API is not supported in this environment
  at Object.Module._extensions..node (node:module:790:21)
  at Module.load (node:module:655:32)
  at Function.Module._load (node:module:523:13)
  at Module.require (node:module:674:19)
  at require (node:module:801:16)
  at Object.<anonymous> (file:///node_modules/.deno/onnxruntime-node@1.20.1/r
  at Object.<anonymous> (file:///node_modules/.deno/onnxruntime-node@1.20.1/r
  at Module._compile (node:module:736:34)
  at Object.Module._extensions..js (node:module:757:11)
  at Module.load (node:module:655:32)
```

I'm not sure what the issue here is, but I hope this issue gets resolved in the future.

## Conclusion

In conclusion, deploying an ONNX-based LLM router using Deno offers a powerful and efficient solution for query routing in production environments. The combination of Deno's modern runtime features — including built-in TypeScript support, secure defaults, and ES Module compatibility — with ONNX's optimized model serving capabilities creates a robust foundation for AI-powered applications. Our implementation demonstrates how to achieve fast inference times (<5ms overhead on top of OpenAI embedding latency) while maintaining the flexibility to support multiple routing models through a clean API interface.

The approach we've outlined not only provides a practical solution for routing between different language models but also showcases the growing maturity of JavaScript-based ML deployment options. By leveraging OpenAI's embeddings and ONNX's cross-platform compatibility, we've created a system that can be easily adapted and scaled based on specific use cases. Whether you're building a cost-optimization layer for LLM API calls or developing a sophisticated query routing system, this architecture provides a solid starting point.

As the field of AI continues to evolve, having efficient and maintainable deployment solutions becomes increasingly critical. The combination of Deno's modern features, ONNX's optimization capabilities, and a well-structured API design pattern offers a blueprint for building production-ready AI services that can grow with your needs.

## About Me

I'm currently a Machine Learning Engineer at Meta, working on ways to improve the users' experience on our ads products. Before that, I was a Machine Learning Engineer at TELUS and a Ph.D. graduate from the University of Alberta. What I wrote here is a result of my personal interest in this space and a way to organize my thoughts and findings.

In case you want to know more about me, check out my [website](#). The views expressed on this account are mine alone and do not necessarily reflect the views of my employer.

Llm

Deno

Onnx

OpenAI

Genai



Follow

## Published in ITNEXT

70K Followers · Last published 10 hours ago

ITNEXT is a platform for IT developers & software engineers to share knowledge, connect, collaborate, learn and experience next-gen technologies.



Follow

## Written by Nima Mahmoudi

210 Followers · 123 Following

I'm currently a Machine Learning Engineer at Meta, working on ways to improve the users' experience on our ads products.

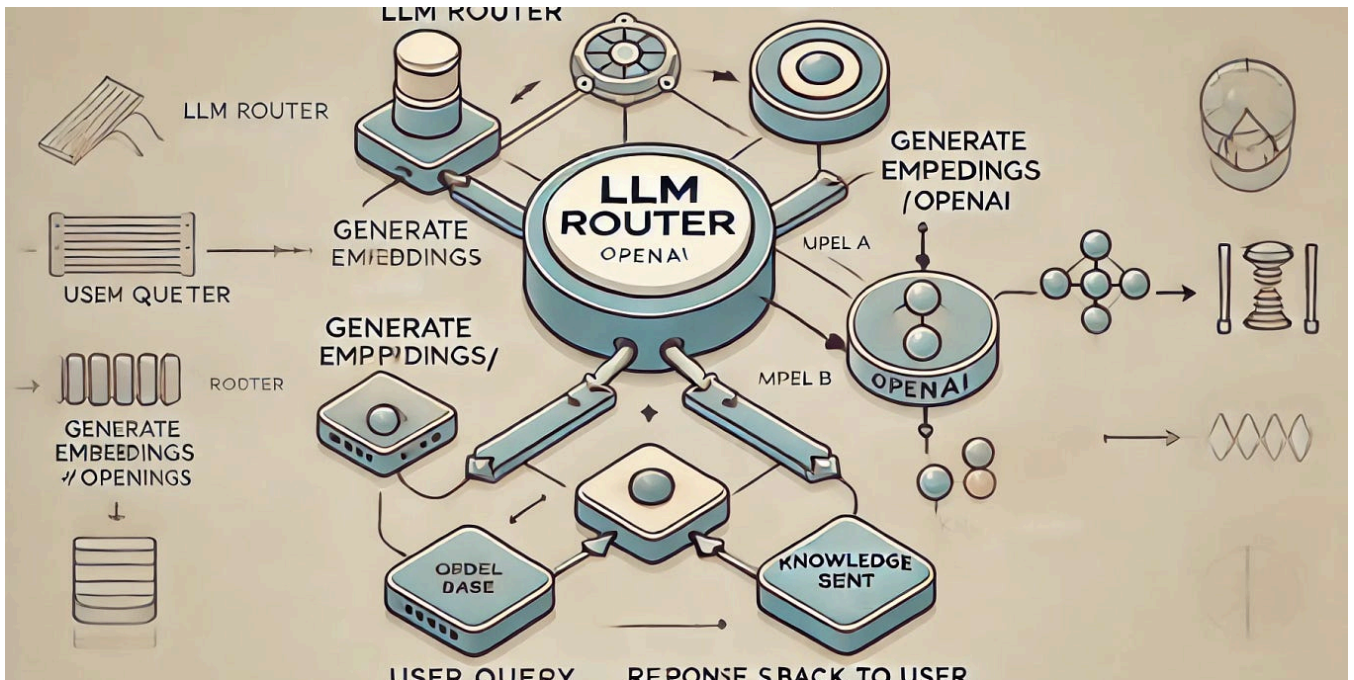
## No responses yet



qsbye

What are your thoughts?

## More from Nima Mahmoudi and ITNEXT



ⓧ In ITNEXT by Nima Mahmoudi

## Building an LLM Router using OpenAI Embeddings

Image generated by DALL.E

★ Dec 31, 2024 🖱 171

🔖 ...



ⓧ In ITNEXT by Zsolt Deak

## Are you a coder? Be an engineer instead!

Especially if you're afraid of AI taking your job

★ Mar 7 🖱 338 💬 20

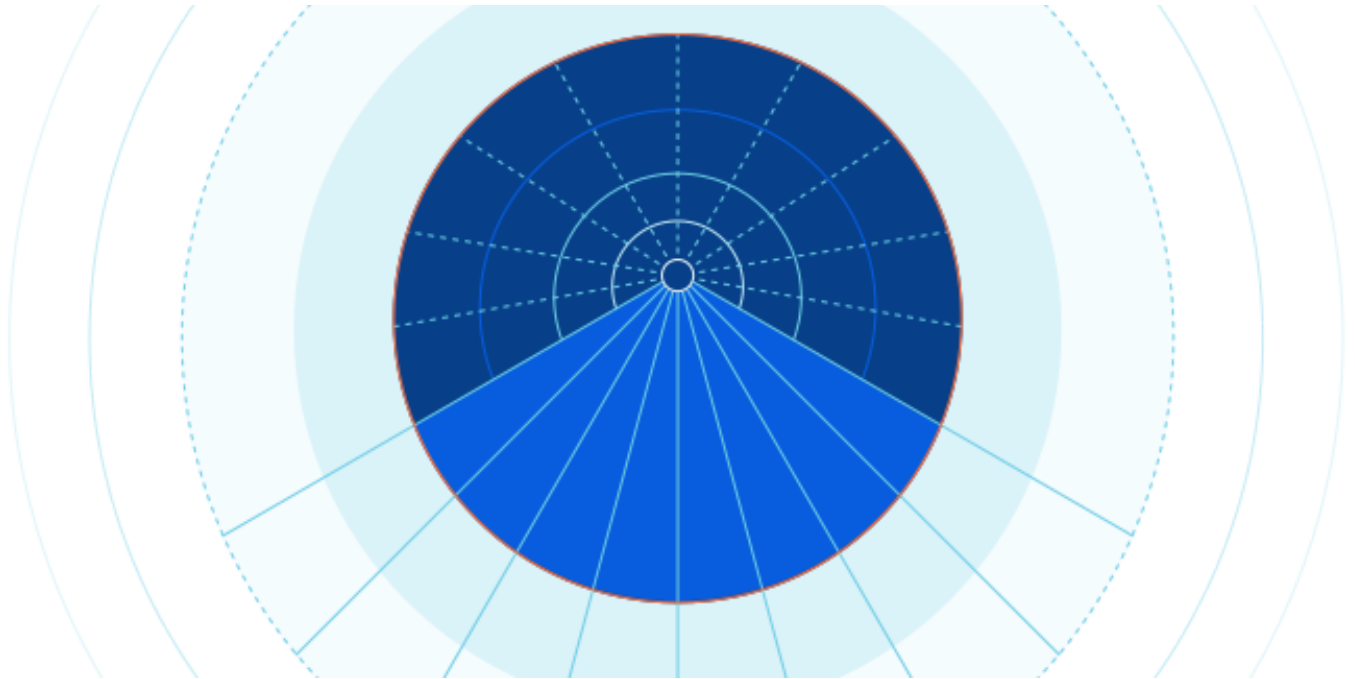


```
1 const state = new Proxy({ count: 0 }, {  
2   set(target, key, value) {  
3     console.log(`${key} changed to ${value}`);  
4     target[key] = value;  
5     return true;  
6   }  
7 });  
8  
9 state.count++; // count changed to 1  
10 state.count++; // count changed to 2
```

ⓧ In ITNEXT by Maxime

## How JavaScript Proxies Fixed My Messy Frontend Code (and Saved My Sanity)

Stop writing the same boilerplate



ⓧ In ITNEXT by Nima Mahmoudi

## Using Cloudflare Tunnels to Securely Expose Kubernetes Services

Cloudflare Tunnel (previously known as Argo Tunnel) is a tool that allows a private and secure connection between your web server and...

Jan 26, 2022 🖱 89 💬 4

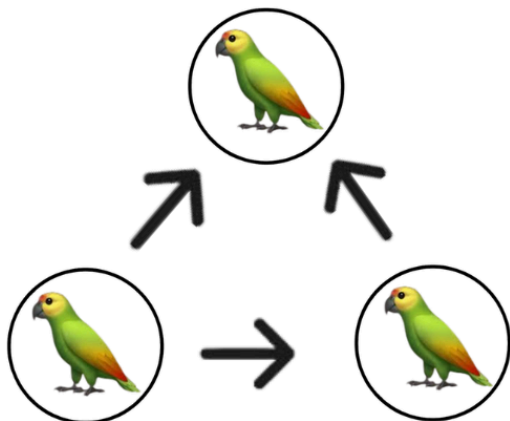
🔖+ ⋮

See all from Nima Mahmoudi

See all from ITNEXT

Recommended from Medium

# LANGGRAPH MCP



## IS INSANE

 In Data Science Collective by Gao Dalie (高達烈)

### LangGraph + MCP + Ollama: The Key To Powerful Agentic AI

In this story, I have a super quick tutorial showing you how to create a multi-agent chatbot using LangGraph, MCP, and Ollama to build a...

★ Mar 29 🖱 940 💬 6



 In Dev Simplified by Neha Gupta

### Why Companies Are Saying GoodBye to Next.js?

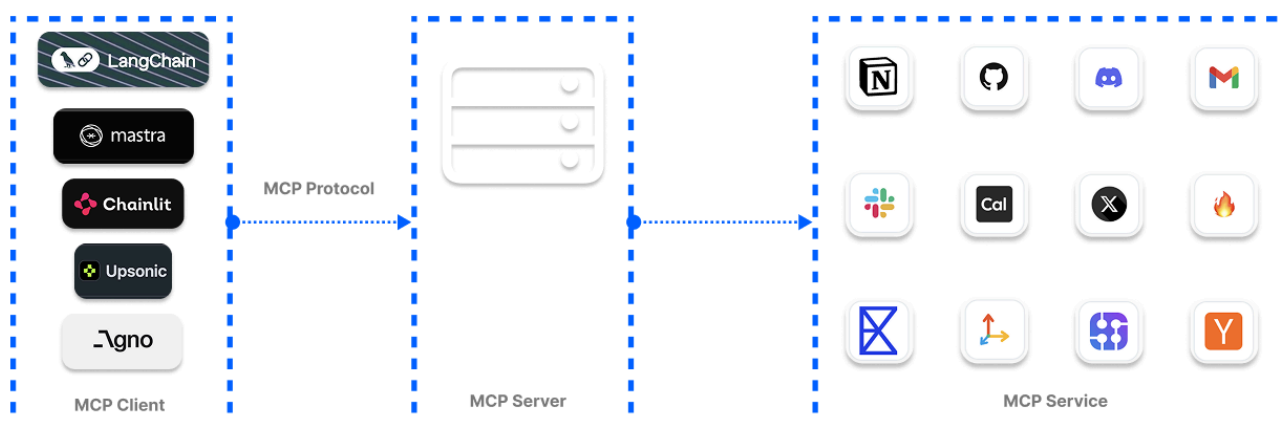
Are you using Next.js or planning to for your next project? Then you need to know this before making a decision!



In Everyday AI by Manpreet Singh

## Craziest MCP Servers You Must Try

I remember when I first heard about MCP (Model Context Protocol). I thought

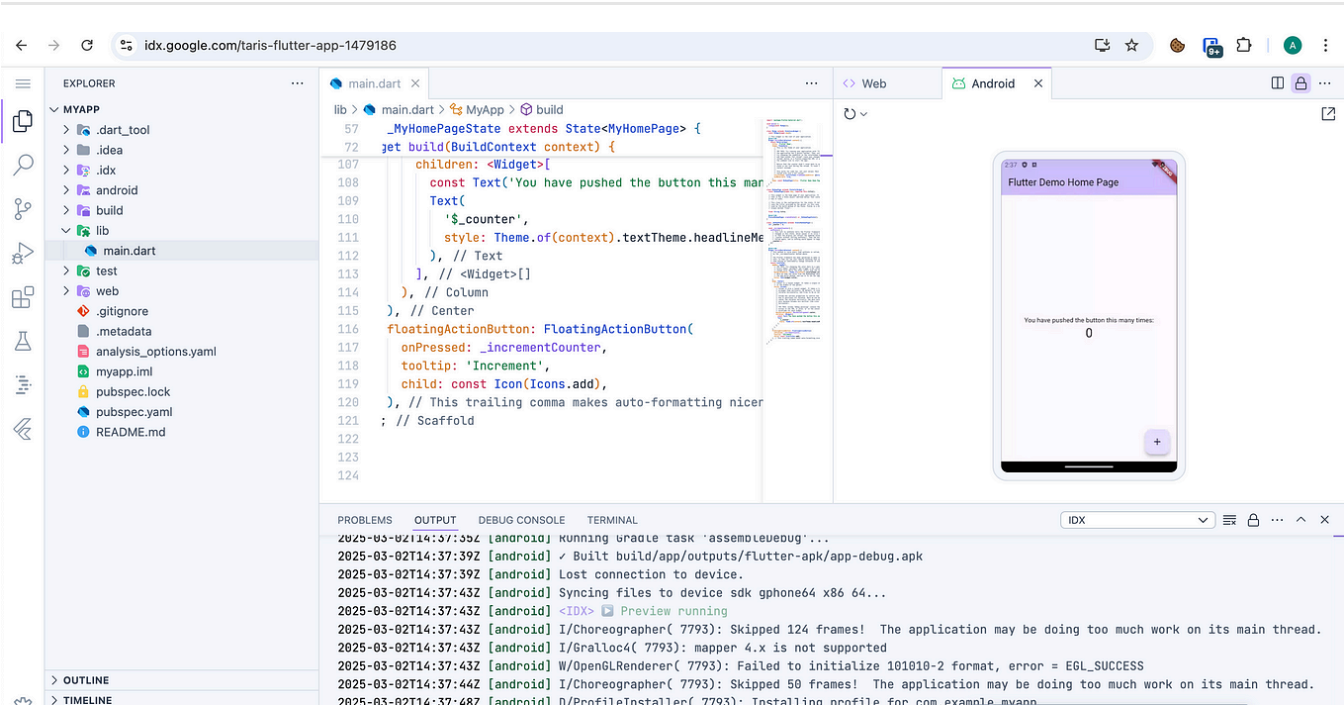


Amos Gyamfi

## The Top 7 MCP-Supported AI Frameworks

Create AI apps with Python and Typescript frameworks that leverage MCP servers to provide context to LLMs.

Apr 1 678 16

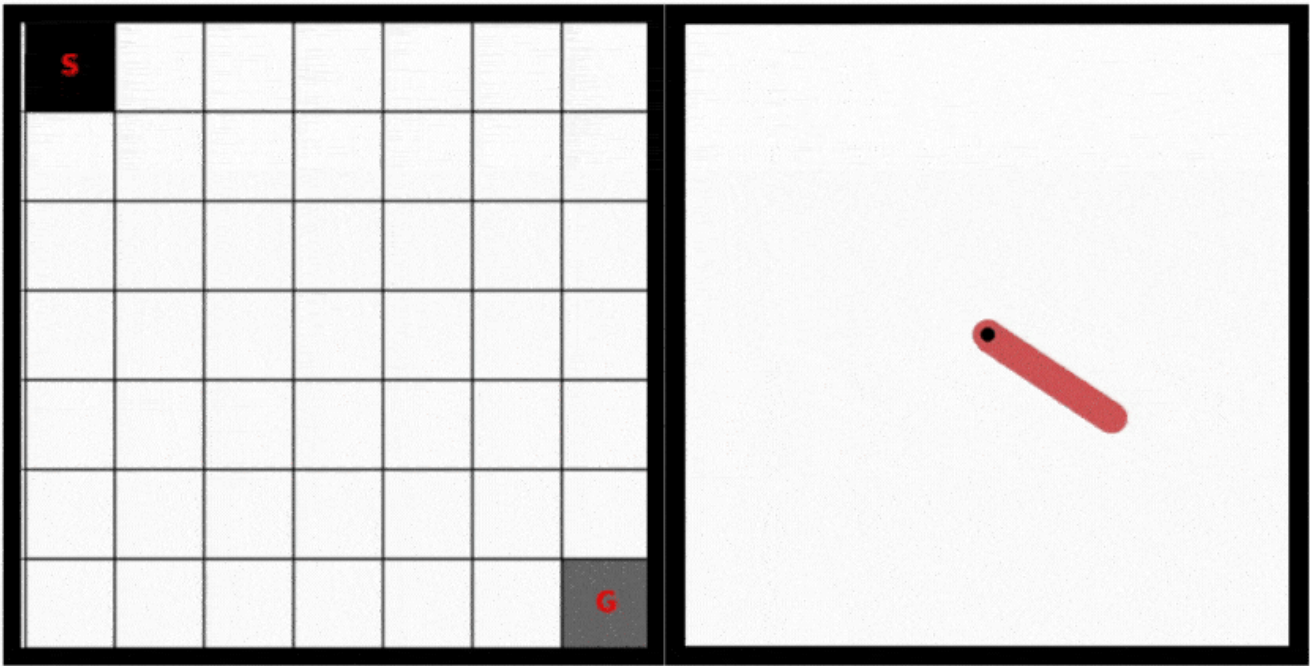


In Coding Beauty by Tari Ibaba

## This new IDE from Google is an absolute game changer

This new IDE from Google is seriously revolutionary.

Mar 12 3.8K 206





In Level Up Coding by Fareed Khan

## Drawing and Coding 18 RL Algorithms from Scratch

PPO, A3C, PlaNet and more!



5d ago



380



2



See more recommendations